

JWT Authentication in the Tea Collection Management System

Harboost Factory Web Portal — Backend Notes

2026-07-27

JWT Authentication in this System

It's a stateless, token-based flow: the NestJS backend (`@nestjs/jwt + passport-jwt`) issues a signed JWT on login; the React portal stores it and attaches it to every subsequent request. No sessions, no cookies.

1. Login — issuing the token

`POST /auth/login` (`auth.controller.ts:12-15`) calls `AuthService.login()` (`auth.service.ts:29-49`):

1. Looks up the user by **phone** (not email — rural mobile users), via `userService.findByPhone(phone)`.
2. Verifies the password with `bcrypt.compare()`.
3. Maps the DB role (`factory_admin` / `factory_officer` / `factory_manager`) to the portal's `AppRole` (`Administrator` / `Officer` / `Manager`) via `role-map.ts`. Any other DB role (estate owner, collector, etc.) is rejected with 401, since only factory-side roles use this portal.
4. Signs a payload of just `{ sub: userId, role: AppRole }` (`jwt-payload.interface.ts`) — deliberately minimal, no name/phone baked in, so a name change doesn't require re-login.
5. Returns `{ accessToken, user }` to the client.

The signing secret and expiry come from environment variables via `JwtModule.registerAsync` in `auth.module.ts:15-27` (`JWT_SECRET`, `JWT_EXPIRES_IN`, default 7 days).

2. Client — storing and attaching the token

`AuthContext.login()` (`AuthContext.tsx:63-70`) calls the login endpoint, then `setToken(accessToken)` stores it in `localStorage` under the key `harboost.token`. Every API call after that goes through `apiFetch()` (`lib/api.ts:37-45`), which reads that token and sets `Authorization: Bearer <token>` on every request automatically — individual `services/*.ts` files never touch the token directly.

On app load, `AuthContext` (`AuthContext.tsx:35-52`) checks for a stored token and calls `GET /auth/me` to rehydrate the session, so a page refresh doesn't log the user out.

3. Protecting a route — verifying the token

Any controller that needs auth adds `@UseGuards(JwtAuthGuard)` (`jwt-auth.guard.ts` — a one-line Passport wrapper). Under the hood, `JwtStrategy` (`jwt.strategy.ts:8-19`) pulls the bearer token from the header, verifies the signature and expiry against `JWT_SECRET`, and if valid, attaches

the decoded payload as `req.user`. Every business controller (Fertilizer, Reports, Collections, etc.) is guarded this way.

If the token is missing, expired, or invalid, the request fails with **401**, and the frontend's `apiFetch` detects the 401 and calls `onUnauthorized()`, which is wired to `AuthContext`'s `logout()` — so an expired token auto-logs-out the user (`AuthContext.tsx:59-61`).

4. Using the identity — the `resolveActor` pattern

The JWT only carries `{ sub, role }` — no name. Every write-path controller resolves the full actor once per request, e.g. `FertilizerController.resolveActor()` (`fertilizer.controller.ts:122-125`):

```
private async resolveActor(req: AuthedRequest): Promise<Actor> {
  const profile = await this.userService.getProfile(Number(req.user.sub));
  return { name: profile?.name ?? 'Unknown', role: req.user.role };
}
```

This gives the service layer a `{ name, role }` to stamp on records (`enteredBy`, `recordedBy`, `decidedBy`) and to authorize with — e.g. `ReportsService.assertCanWrite()` throws 403 if `actor.role === 'Manager'`.

Worked example — logging a daily expense

1. POST `/auth/login` `{ "phone": "0771234568", "password": "Password123!" }`
→ `{ "accessToken": "eyJhbGci...", "user": { "role": "Officer", "name": "S. Fernando", ... } }`
2. Frontend: `setToken(accessToken)` // saved to `localStorage`
3. POST `/reports/expenses`
Header: `Authorization: Bearer eyJhbGci...`
Body: `{ "category": "Utilities", "amount": 1500, "date": "2026-07-20", "description": "..."}`
4. Backend:
`JwtAuthGuard` → `JwtStrategy` verifies signature → `req.user = { sub: "5", role: "Officer" }`
`ReportsController.resolveActor(req)` → `{ name: "S. Fernando", role: "Officer" }`
`ReportsService.createExpense(dto, actor)` → `assertCanWrite` passes (not Manager) → row saved
`enteredBy: "S. Fernando"`
→ 201 `{ "id": "EXP-2026-0006", "enteredBy": "S. Fernando", ... }`

If the same request were sent with a **Manager**'s token, `req.user.role` would be `"Manager"`, `assertCanWrite()` would throw `ForbiddenException`, and the response would be **403** — the same behavior verified live against the running backend during the Reports slice work (`reports.service.ts, assertCanWrite`).

Key design points worth remembering

- **Stateless** — no server-side session store. The signature alone proves validity, so any backend instance can verify any token without a shared session cache.
- **Role lives in the token, not re-fetched per request** — `req.user.role` is trusted directly for permission checks (e.g. `assertCanWrite`), avoiding a DB round-trip just to check role.

Name/profile *is* re-fetched (`resolveActor`) since it's needed for stamping and isn't worth bloating the token for.

- **No refresh-token flow yet** — a 7-day token simply expires and forces re-login; this is a known simplification, not yet built out.